GitHub Copilotで
既存リポジトリを段階的に改善する

テスト・リンター・CI/CDをCopilotで導入する実践手順

© 2026

本セミナーの流れ

#	内容	ゴール
0	プロンプトの書き方	ワンショット・Fewショットの違いを理解する
1	テストの導入	既存コードを変えずにテストを追加できる
2	リンターの導入	決定論的ツールでAI出力のブレを補える
3	CI/CDの構築	変更のたびに自動検証が走る状態を作れる
4	Copilot指示書	プロジェクト固有の文脈をAIに伝えられる

ゴール： Copilotを使って既存リポジトリの品質基盤を段階的に構築できるようになる

© 2026

効果的なプロンプトの書き方：ワンショットとFewショット

ワンショット — 指示だけを渡す。シンプルだが文脈不足になりやすい

```
FlaskアプリのAPIテストをpytestで書いてください
```

Fewショット（例示付き） — 入出力の具体例を一緒に渡す。出力精度が上がる

```
以下のテストを参考に、DELETEエンドポイントのテストを追加してください

【例】
def test_タスクを正常に作成(client):
    res = client.post("/tasks", json={"title": "テスト"})
    assert res.status_code == 201
```

例示があるほど、AIは期待する形式・命名・粒度を正確に再現できる

© 2026

今回の手法：ブランチ差分をFewショットとして活用する

本セミナーでは **ブランチ間の差分** を「例示」として渡す：

```
git@github.com:sotanengel/gitHub-copilot-first-try.git の
step/1-api → step/2-tests ブランチの差分を参考に、
同様の変更を段階的に実装してください
```

なぜこの方法が効くのか？

- 差分には「何を」「どこに」「どう」変更したかが全て含まれる
- ワンショットの手軽さで、Fewショット相当の精度が得られる
- 各ステップの差分が独立しているため、段階的に適用できる

© 2026

サンプルアプリの概要

FlaskベースのタスクAPI — シンプルなCRUDアプリケーション

メソッド	パス	説明
GET	`/tasks`	全タスク一覧取得
POST	`/tasks`	タスク作成
PUT	`/tasks/<id>`	タスク更新
DELETE	`/tasks/<id>`	タスク削除
PATCH	`/tasks/<id>/toggle`	完了/未完了トグル
GET	`/tasks/search?q=`	タスク検索

技術スタック：Python 3.10+ / Flask / SQLite / uv

© 2026

ブランチ構成：各ステップが独立したブランチになっている

```
main
  └─ step/1-api                      ← 素のFlask API（テスト・リンターなし）
       └─ step/2-tests               ← テスト追加（22テスト）
            └─ step/3-linter         ← リンター導入（ruff）
                 └─ step/4-cicd      ← CI/CD追加（GitHub Actions）
                      └─ step/5-copilot-instructions ← Copilot指示書
```

各ブランチの差分で「何が追加されたか」が一目瞭然

→ この差分をプロンプトの「例示」として使う

© 2026

テストもリンターもないコードは変更が怖い

```
# app.py - FlaskベースのタスクAPI
from flask import Flask, request, jsonify
import sqlite3
import os   # ← 未使用のimport

@app.route('/tasks', methods=['GET'])
def get_tasks():
    conn = sqlite3.connect('tasks.db')  # テストなし、CIなし
```

テストなし / リンターなし / CI/CDなし → **変更するたびに不安が生まれる**

© 2026

Step 1：既存コードを変えずにテストだけ追加する

`conftest.py` / `test_api.py`（18件） / `test_scenarios.py`（4件）を生成
既存コードは **一切変更しない** — コード文脈を理解した生成がここで活きる

プロンプト例：

```
git@github.com:sotanengel/gitHub-copilot-first-try.git の
step/1-api ブランチから step/2-tests ブランチの差分を参考に、
テストを段階的に実装してください
```

© 2026

22テストが全てパス — 既存コード変更なし

```
test_api.py::test_タスクを正常に作成          PASSED
test_api.py::test_タスクを正常に削除          PASSED
test_api.py::test_タスクを検索                PASSED
test_scenarios.py::test_作成_更新_完了_削除フロー PASSED
========================= 22 passed in 0.21s ====
```

既存コードの変更なしにテストカバレッジを確保できた

© 2026

「決定論」と「確率論」— このスライドでの意味

決定論（deterministic）
辞書的意味：同じ条件なら必ず同じ結果になるという考え方
このスライドでは：リンターやCIのように **同じ入力に対して毎回同じ結果** を返すツール

確率論（probabilistic）
辞書的意味：結果が確率的に変動するという考え方
このスライドでは：Copilotのように **同じプロンプトでも出力が毎回異なりうる** AI

AIの出力ブレを決定論的ツールで検証する — これがStep 2以降のテーマ

© 2026

Step 2：決定論的なリンターでAIの出力ブレを補う

Copilotでruff設定を生成 → **ルールベースで一貫した整形** を実行
AIの生成は確率的 → リンターが決定論的に統一する

プロンプト例：

```
git@github.com:sotanengel/gitHub-copilot-first-try.git の
step/2-tests ブランチから step/3-linter ブランチの差分を参考に、
リンター設定を段階的に導入してください
```

© 2026

Step 3：CIで「確率論的な変更」を毎回検証する

AIが生成したコードも人が書いたコードも、同じ基準で自動検証される
この仕組みがあるからこそ、AIを安心して活用できる

プロンプト例：

```
git@github.com:sotanengel/gitHub-copilot-first-try.git の
step/3-linter ブランチから step/4-cicd ブランチの差分を参考に、
CI/CDワークフローを段階的に構築してください
```

© 2026

Step 4：指示書でCopilotの生成精度を底上げする

確率論的なAIに文脈を与えることで、生成の的中率が上がる
プロジェクト概要 / コーディング規約 / 注意事項を `.github/copilot-instructions.md` に記述

プロンプト例：

```
git@github.com:sotanengel/gitHub-copilot-first-try.git の
step/4-cicd ブランチから step/5-copilot-instructions ブランチの差分を
参考に、Copilot指示書を作成してください
```

© 2026

4ステップで品質基盤が整った

項目	Before	After
テスト	なし	22テスト（結合＋連結）
リンター	なし	ruff（lint + format）
CI/CD	なし	GitHub Actions
Copilot指示書	なし	copilot-instructions.md
品質管理	手動	自動化

© 2026

まとめ：確率論的AIと決定論的ツールを組み合わせる

Copilot（確率論的） — コード文脈を理解し、テスト・設定・CIを生成
出力にばらつきはあるが、指示書で精度を底上げできる

リンター・CI（決定論的） — ルールに基づき毎回同じ基準で検証
AIの出力ブレを吸収し、品質を一定に保つ

両者の組み合わせが鍵
AIで素早く生成し、決定論的ツールで確実に検証する
このサイクルが、既存コードを壊さず段階的に改善する土台になる

© 2026

Tips: CopilotはPR文も自動生成できる

1. PRテンプレートと指示書を用意しておく
2. 開発完了後にCopilotへ一言依頼する

> 「このブランチの変更内容を元に、PRテンプレートに沿ったPR文を作成して」

Copilotが差分を読み取り、概要・変更内容・テスト項目を自動で埋める

© 2026

Tips: ブランチ保護もコードで管理すれば変更履歴が残る

1. `branch-protection.json` に保護ルールをJSONで定義
2. mainマージ時にGitHub Actionsが自動適用
3. ルール変更はJSON編集 → PR → マージだけ

PRレビュー必須 / ステータスチェック必須 / Force Push禁止 / 管理者にも適用

© 2026

Tips: 契約プログラミングで関数の仕様を明示しAI精度を上げる

```
@contract(
    pre(lambda task: task is not None),
    post(lambda result: "id" in result),
    pure(),
)
def task_to_dict(task):
    return {"id": task["id"], "title": task["title"], ...}
```

- `pre()` / `post()` / `pure()` で関数の意図をコードで宣言
- Copilotが仕様を正確に理解し、生成コードの品質が向上

© 2026

Tips: サプライチェーン攻撃を3層で防ぐ

GitHub Actions SHA pinning — ActionをコミットSHAで固定

```
- uses: actions/checkout@34e1148...  # v4
```

uv exclude-newer — 公開3日未満のパッケージを除外

Dependabot + cooldown — 依存更新自動化 + 3日の検疫期間

© 2026

Tips: AIエージェントはセキュアなサンドボックスで動かす

```yaml
# compose.yaml
services:
  sandbox:
    read_only: true        # root FSは読み取り専用
    network_mode: none     # オフライン既定
    cap_drop: [ALL]        # 全権限をdrop
```

- オフライン既定 → 依存取得時のみ `--online` で明示許可
- 書き込みは `/workspace` と `.sandbox/home` のみ
- 監査ログをhost側に記録

© 2026